

Building elegant web experiences with a powerful backend API and polished frontend design.

Explore a dynamic portfolio that demonstrates modern JavaScript, Express.js services, MongoDB data integration, responsive UX, and practical deployment readiness.

Express	MongoDB	JavaScript
API Server	Database Store	Interactive UI

ABOUT ME

A full-stack developer ready for internships and modern product development.

This portfolio communicates real backend integration, database-driven content, and frontend usability. It includes a live Express API, a MongoDB dataset, and a polished responsive interface.

SKILLS

Tools and technologies I use to build modern full-stack apps.

JavaScript	Advanced	Node.js	Express	MongoDB	Database
HTML & CSS	Responsive	REST API	Backend	Deployment	Ready

FEATURES

A portfolio packed with real-world features and modern UX.

1. Live API Integration

Projects load from an Express endpoint and render dynamically on the page.

2. Category Filtering

Choose tags like Frontend, Backend, Database, UI/UX, or DevOps to filter project work.

3. Contact Form + Backend Storage

Messages are submitted to the server and stored in MongoDB for review.

4. Responsive Design

Layouts adapt for desktop, tablet, and mobile screens.

5. Theme Switcher

Toggle between dark and light themes for better reading and presentation.

6. Search-Enabled Project Browser

Search projects instantly by title or description.

7. Favorite Projects

Save favorite projects locally so they stand out during review.

8. Modal Project Details

View complete project metadata and repo links in a polished modal overlay.

9. Live Summary Statistics

See real-time project count, message count, and uptime in the hero section.

10. Category Landing Sections

Dedicated sections highlight Frontend, Backend, Database, UI/UX, and DevOps work.

PROJECTS

Dynamic portfolio work driven by live project data.

Portfolio Website

[\[Frontend\]](#) [\[Backend\]](#) [\[Database\]](#)

A modern full-stack portfolio website built with Express, MongoDB, and React-driven frontend architecture.

Repo: github.com/your-username/portfolio

Task Tracker App

[\[Frontend\]](#) [\[UI/UX\]](#) [\[API\]](#)

A lightweight task manager with filters, deadlines, and a responsive interface.

Repo: github.com/your-username/task-tracker

Landing Page Redesign

[\[Frontend\]](#) [\[UI/UX\]](#) [\[Design\]](#)

A polished frontend landing page built for conversion, accessibility, and responsive UX.

Live: example.com/landing-page

Repo: github.com/your-username/landing-page-redesign

Blog Dashboard

[\[Backend\]](#) [\[Dashboard\]](#) [\[DevOps\]](#)

An admin dashboard designed for content planning, analytics, and publishing workflows.

Repo: github.com/your-username/blog-dashboard

API Services Suite

[\[Backend\]](#) [\[Database\]](#) [\[DevOps\]](#)

A backend service layer that powers frontend applications and stores data in MongoDB.

Repo: github.com/your-username/api-services

Backend routes exposed by the Express server.

GET	/api/projects	Returns all project data
GET	/api/projects?tag=X	Returns projects filtered by tag
GET	/api/summary	Returns project count, message count, server uptime
POST	/api/contact	Stores contact messages in MongoDB
GET	/api/profile	Returns profile metadata
POST	/api/admin/login	Verify admin password
POST	/api/projects	Create a new project (admin only)
PUT	/api/projects/:id	Update a project (admin only)
DELETE	/api/projects/:id	Delete a project (admin only)
GET	/api/messages	List contact messages (admin only)

SERVER CODE

server.js — Express + MongoDB backend

```
const express = require('express');
const cors = require('cors');
const mongoose = require('mongoose');
const dotenv = require('dotenv');
dotenv.config();

const app = express();
app.use(cors());
app.use(express.json());

// MongoDB schemas
const projectSchema = new mongoose.Schema({
  title: { type: String, required: true },
  description: { type: String, required: true },
  tags: { type: [String], default: [] },
  link: String, repo: String, image: String, order: Number
}, { timestamps: true });

const messageSchema = new mongoose.Schema({
  name: { type: String, required: true },
  email: { type: String, required: true },
  subject: String,
  message: { type: String, required: true }
}, { timestamps: true });
```

```
const Project = mongoose.model('Project', projectSchema);
const Message = mongoose.model('Message', messageSchema);

// GET /api/projects (with optional ?tag= filter)
app.get('/api/projects', async (req, res) => {
  const { tag } = req.query;
  const filter = tag && tag !== 'All' ? { tags: tag } : {};
  const projects = await Project.find(filter).sort({ order: 1 });
  res.json(projects);
});

// POST /api/contact - stores message in MongoDB
app.post('/api/contact', async (req, res) => {
  const { name, email, subject, message } = req.body;
  if (!name || !email || !message)
    return res.status(400).json({ message: 'Required fields missing.' });
  await Message.create({ name, email, subject, message });
  res.json({ message: 'Message received.' });
});

mongoose.connect(process.env.MONGODB_URI || 'mongodb://127.0.0.1:27017/portfolio')
  .then(() => app.listen(process.env.PORT || 5000));
```

EXPERIENCE

How this website shows practical full-stack workflow.

● Backend Architecture

Express routes, MongoDB models, valid JSON APIs, query-based filtering, and production-ready middleware.

● Data-Driven Projects

Projects are loaded from MongoDB and rendered dynamically for a real portfolio experience.

● Contact Backend

User messages are posted to the backend and stored in the database for later review.

● Responsive UI

Designed to look great across desktop, tablet, and mobile screens.

CONTACT

Ready to discuss your next internship or web development project.

Email	ramya.dev@example.com
Location	Remote / open to internship opportunities
Availability	Available for internships starting May 2026

